Unité 3 — Heap & erreurs temporelles

Livrable B — Support de cours · UE « Software Security »

Code UE 7532 14 U32 D1 · 11 octobre 2026

> **Place dans le programme.** Après la violation *spatiale* sur la pile (Unité 2), cette unité traite les erreurs du **tas** et les erreurs *temporelles* : débordement de tas, **use-after-free**, **double-free**, lecture non initialisée, et les notions d'aliasing qui les sous-tendent. *Contenu fondé sur meltypot (deck « Software Security » : Heap Buffer Overflow, Use-After-Free + visualisation, Double-Free, Aliasing, Heartbleed).*

1. Rappel : sûreté spatiale et temporelle

Un accès mémoire est sûr s'il est **spatialement** valide (dans les bornes de l'objet) **et temporellement** valide (l'objet est encore vivant). L'Unité 2 a montré la violation spatiale sur la pile ; ici, on ajoute les violations **temporelles** (objet déjà libéré) et les violations spatiales sur le **tas**.

2. Le tas

`malloc` / `free` gèrent une zone où les objets de durée de vie dynamique sont placés, souvent **adjacents**, entrelacés de **métadonnées** de l'allocateur. Deux conséquences : un débordement corrompt l'objet voisin ou ces métadonnées ; une libération mal gérée corrompt les listes internes de l'allocateur.

3. Débordement de tas (heap buffer overflow)

C — MELTYPOT « HEAP BUFFER OVERFLOW »

```c
char *user = malloc(8);
char *admin_flag = malloc(1);   // allocation adjacente
strcpy(user, "AAAAAAAAAAA");    // 12 octets dans 8 -> deborde
// admin_flag vaut maintenant 'A' (vrai !)
```

Le débordement d'un objet écrase silencieusement le contenu ou les métadonnées du suivant — souvent un pointeur de fonction, une vtable, ou un drapeau de sécurité.

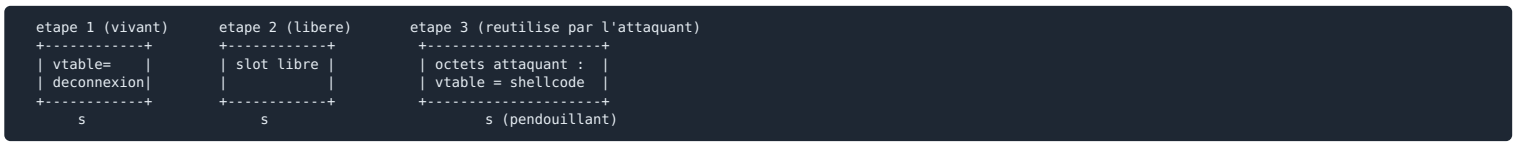
4. Use-after-free (erreur temporelle)

C — MELTYPOT « USE-AFTER-FREE »

```c
typedef struct { void (*action)(void); } Session;
Session *s = malloc(sizeof *s);
s->action = deconnexion;
free(s);              // memoire rendue a l'allocateur
// ... s est desormais un pointeur "pendouillant" ...
s->action();          // execute du code arbitraire si le slot a ete reutilise
```

Schéma d'exploitation (meltypot) : (1) libérer l'objet victime ; (2) déclencher une allocation de même taille pour que l'allocateur **rende le même slot**, rempli d'octets contrôlés (fausse vtable / faux pointeur de fonction) ; (3) attendre que le pointeur pendouillant soit déréférencé.

SCHÉMA — UAF VISUALISÉ (MELTYPOT)



5. Double-free

C — MELTYPOT « DOUBLE-FREE »

```c
char *p = malloc(32);
free(p);
/* ... chemin d'erreur ... */
free(p);     // boum : corrompt les listes internes de l'allocateur
```

Le second `free` corrompt les listes chaînées de l'allocateur. Les exploits « unlink » classiques de la glibc en tiraient parti pour faire renvoyer à `malloc` une adresse arbitraire. Cause fréquente : chemins d'erreur qui nettoient deux fois, ou propriété d'un pointeur mal définie entre deux modules.

6. Lecture de mémoire non initialisée

Lire une variable (pile ou tas) avant toute écriture renvoie des octets indéterminés : comportement non défini, résultats non déterministes, et parfois **fuite d'informations** (des octets résiduels de la mémoire sont exposés).

7. Aliasing : la cause profonde

Deux noms (pointeurs, références) sont en **aliasing** s'ils désignent la même case mémoire. L'aliasing rend difficile de répondre à la question centrale : « qui a le droit de lire/écrire cette case, et quand ? ». Cas classique en C++ (meltypot) : l'**invalidation d'itérateur** :

C++ — INVALIDATION D'ITÉRATEUR (MELTYPOT)

```cpp
std::vector<int> v = {1,2,3,4};
int &first = v[0];     // reference dans le tampon de v
v.push_back(5);        // peut reallouer -> deplace le tampon
std::cout << first;    // reference pendouillante : comportement non defini
```

Le C offre `restrict` pour *promettre* l'absence d'aliasing (optimisation), mais cette promesse n'est **pas vérifiée** : mentir produit du code erroné. C'est précisément ce que le système de types de Rust vérifie à la compilation (Unité 6).

8. Heartbleed : un vrai bug mémoire

C — MELTYPOT « HEARTBLEED »

```c
/* OpenSSL 1.0.1 (2012), simplifie */
memcpy(response, request->payload, request->length);   // length fourni par l'attaquant !
```

La longueur était contrôlée par l'attaquant : en demandant `length = 65535` pour quelques octets réels, le serveur recopiait jusqu'à 64 Ko de mémoire voisine (mots de passe, cookies, clés privées). **Un seul contrôle de borne manquant**, ~17 % des serveurs TLS exposés.

9. Détection et remédiation

- **AddressSanitizer** détecte use-after-free, double-free, débordements de tas/pile, avec fichier et ligne (labos C).
- **Valgrind (memcheck)** détecte en plus les lectures non initialisées.
- **Remédiations** : mettre les pointeurs à `NULL` après `free` ; propriété claire d'un pointeur (un seul propriétaire) ; contrôle systématique des bornes ; initialiser à l'allocation ; langages sûrs en mémoire (Unité 6).

10. Points clés

- Le tas place les objets côte à côte : un débordement corrompt le voisin ou les métadonnées.
- UAF et double-free sont des erreurs *temporelles* : le « quand » compte autant que le « où ».
- L'aliasing est la cause profonde ; `restrict` promet sans vérifier.
- ASan + Valgrind transforment ces bugs silencieux en diagnostics précis.

Exemples conceptuels repris de meltypot ; programmes de labo et sorties ASan/Valgrind produits et vérifiés (clang-17 / Valgrind 3.22).